El paquet `pxLST`

Llistats de codi font professionals per a LaTeX2e i P3CTeX

Scvria
P3CTeX / UOC PEC Repository

2026/10/27 – Versió 0.2

Resum

`pxLST` proporciona una interfície LaTeX2e completa, estable i ergonòmica per a la presentació de fragments i fitxers de codi font amb ressaltat sintàctic professional basat en `listings` i emmarcat en caixes `tcolorbox`. Està especialment dissenyat per a les assignatures d'Enginyeria Informàtica de la UOC (Compiladors, Sistemes Encastats, Algorísmica, Bases de Dades, Ciència de Dades, etc.), oferint suport natiu per a 17 llenguatges i accents catalans amb paletes d'estil adaptades tant a pantalla com a impressió.

Índex

1 Càrrega i activació

En documents basats en la classe `P3CTeX`, el mòdul `pxLST` es carrega automàticament en el paquet per defecte:

```
\documentclass[PAC,default]{P3CTeX}
```

També es pot emprar en qualsevol classe de document LaTeX estàndard (com ara `article` o `report`):

```
\usepackage{pxLST}
```

2 Configuració global

2.1 `\pxLSTsetup` i `\pxCodeSet`

La instrucció `\pxLSTsetup` (o el seu sinònim `\pxCodeSet`) permet establir la paleta de colors activa, el llenguatge de programació per defecte i l'estil de llistat:

```
\pxLSTsetup[<paleta>]{<llenguatge>}[<estil>]
```

- `<paleta>`: nom de la paleta de colors (opcional, per defecte `dark`). Les paletes disponibles són `dark`, `pastel` i `light`.
- `<llenguatge>`: nom del llenguatge `listings` predefinit (per exemple `pxJava`, `pxC`, `pxPython`, `pxR`, `algoritme`, etc.).
- `<estil>`: estil base de `listings` (opcional, per defecte `pxlst`). Altres estils suportats són `block`, `framed` i `plain`.

Exemples:

```
\pxLSTsetup[dark]{pxJava}
\pxLSTsetup[pastel]{algoritme}[framed]
\pxLSTsetup[light]{pxR}
```

2.2 Paletes de colors

El paquet incorpora tres paletes acuradament calibrades:

- `dark`: paleta fosca d'alt contrast inspirada en Darcula, ideal per a lectura còmoda en pantalla.
- `pastel`: variant suau de la paleta fosca per a documents electrònics de to moderat.
- `light`: paleta clara amb fons net, optimitzada per a impressió en paper i informes acadèmics formals.

3 Entorns de codi

3.1 `pxCode` i l'àlies `pxCode`

L'entorn principal per a blocs de codi amb títol i números de línia:

```
\begin{pxCode}[<opcions listings>]{<titol>}
    ... codi font ...
\end{pxCode}
```

L'àlies `pxCode` (i `pxCode`) té exactament el mateix contracte d'arguments i és 100% compatible amb versions anteriors:

```
\begin{pxCode}[numbers=none]{Funció suma}
int suma(int a, int b) {
    return a + b;
}
\end{pxCode}
```

3.2 pxCode*

Variant amb un fons lleugerament més fosc per distingir blocs alternatius o seccions secundàries:

```
\begin{pxCode*}[language=pxPython]{Entrada de dades}
def processar(dades):
    return [x * 2 for x in dades]
\end{pxCode*}
```

3.3 pxCodeMini

Entorn compacte sense numeració de línia, dissenyat per a petits fragments de consola o comandes:

```
\begin{pxCodeMini}[.7\linewidth]
#!/bin/bash
git status
pytest -v
\end{pxCodeMini}
```

L'amplada per defecte és `.8\linewidth`.

3.4 pxCodeBox i pxCodeBreak

Entorn avançat amb panell lateral d'anotacions i peu de comentari:

```
\begin{pxCodeBox}[<opcions>]{<etiqueta-id>}{<titol>}{<codi-nota>}{<text explicatiu>}
    ... codi font ...
\end{pxCodeBox}
```

- `<etiqueta-id>`: identificador per a referències creuades. S'insereix automàticament `\label{cdbx:<etique` accessible amb `\ref{cdbx:<etiqueta-id>}`.
- `<titol>`: títol visible a la capçalera superior dreta.
- `<codi-nota>`: identificador curt que apareix a l'etiqueta esquerra.
- `<text explicatiu>`: comentari situat a la safata inferior de la caixa.

La variant `pxCodeBreak` permet salts de pàgina.

4 Instruccions d'entrada i codi en línia

4.1 Codi en línia: \pxCodeInline

Per incrustar fragments de codi dins del flux de text:

```
\pxCodeInline[<llenguatge>]{<code>}[<estil>]
```

Exemples amb la mateixa comanda:

```
El valor de retorn \pxCodeInline{int n = 0;} s'obté mitjançant \pxCodeInline{return n + 1;}.
Comanda de terminal: \pxCodeInline[pxBash]{grep -rn "PATATA" .}.
```

4.2 Inclusió de fitxers externs: \pxCodeFile

Permet incloure un fitxer de codi sencer en una caixa divisible:

```
\pxCodeFile[<llenguatge>]{<fitxer>}[<titol>]
```

Exemples:

```
\pxCodeFile[pxJava]{src/Main.java}[Classe Principal]
\pxCodeFile[pxYAML]{config/app.yaml}[Configuració del sistema]
```

5 Llenguatges suportats

pxLST defineix de sèrie 17 configuracions sintàctiques completes:

pxJava: Java modern amb suport per a `records`, `sealed`, `var`, mètodes de `java.util` i `System`.

pxPython: Python 3 amb paraules clau, tipus natius, decoradors i funcions integrades.

pxC: C estàndard (C99/C11) amb tipus `stdint.h` i funcions clàssiques d'E/S i memòria.

pxSTM32C: C especialitzat per a microcontroladors STM32 i la biblioteca HAL (funcions `HAL_*`, registres i interrupcions per a assignatures com SSEE).

pxASM: Llenguatge d'acoblador ARM i x86 amb registres (`r0`–`r15`, `eax`, `rsp`), mnemònics i directives.

pxJLex: Especificacions de l'analitzador lèxic JLex/Lex (directrius `%class`, `%init`, expressions regulars i codi de retorn de `Symbol`).

pxCUP: Especificacions de l'analitzador sintàctic CUP/Yacc (`terminal`, `non terminal`, regles de producció `::=` i codi d'acció).

pxWorkflow: DSL de fluxos de treball, estats, transicions i màquines de decisió.

pxJavaScript / pxJS: JavaScript modern (ES6+) amb `const`, `let`, funcions fletxa i promeses.

pxSQL: Consultes SQL ANSI (`SELECT`, `JOIN`, `GROUP BY`, `HAVING`, etc.).

pxProlog: Programació lògica amb predicats, fets, regles i operadors `:-`.

pxOWL: Ontologies en sintaxi Manchester i Turtle (`Class:`, `SubClassOf:`, `Prefix:`).

pxTAD: Especificacions formals de Tipus Abstractes de Dades segons els convenis de la UOC.

pxYAML: Fitxers estructurats YAML per a configuracions de projectes.

pxBash: Scripts d'intèrpret d'ordres Unix/Linux.

algoritme: Pseudocodi algorísmic formal en català (`Funcio`, `Si`, `Llavors`, `Mentre`, `Retornar`).

pxR: Llenguatge R per a computació estadística, anàlisi de dades, Machine Learning i Data Mining (funcions `lm`, `predict`, `summary`, biblioteques `ggplot2`, `dplyr`, `caret`).

6 Tractament d'accents i caràcters UTF-8

El paquet gestiona de manera transparent caràcters especials catalans i castellans dins dels blocs de codi: l'ela geminada (`l·l`), vocals accentuades (`à`, `è`, `é`, `í`, `ò`, `ó`, `ú`), ce trencada (`ç`), eñe (`ñ`) i símbols ordinals (`º`, `ª`).

7 Literal code and source ranges

Quotes are rendered upright; dollar signs are literal unless `mathescape=true` is explicitly requested. JLex and CUP quoted strings recognise backslash escapes. Inline code also accepts verbatim delimiters:

```
\pxCodeInline[pxBash]|printf '%s' "$HOME"|
\pxCodeChunk[pxJava]{Main.java}{12}{20}{Validation}
\begin{pxTerminal}{Shell session}
$ echo "$HOME"
/home/student
\end{pxTerminal}
```

The terminal accepts listings options in an optional argument and uses the existing palette. Source ranges are inclusive and preserve line numbers. Inline verbatim cannot be used inside another command's argument.